

Run Optimization-AI on Windows

Step-by-step setup that reproduces the exact same results as the development machine.

Prerequisites

Requirement	Notes
Windows 10 / 11 (64-bit)	
Python 3.10–3.12	From python.org — tick " Add Python to PATH " during install.
Ollama for Windows	From https://ollama.com/download . Runs the local LLM.
~15 GB free disk	The <code>qwen3:14b</code> model is ~9.3 GB.
~16 GB RAM recommended	Model runs in RAM on CPU; a GPU is optional but much faster.

Good news vs. the old instructions: you do **not** need to install GLPK or fiddle with PATH for a solver. The project now uses HiGHS, which installs automatically as a Python wheel (`highspy`) via `pip`. Ignore the GLPK / `qwen2:7b` steps in the old `setup.bat` — this guide supersedes them.

Setup (one time)

1. **Install Ollama**, then in a terminal pull the model:

```
ollama pull qwen3:14b
```

Leave Ollama running (it auto-starts as a background service on Windows).

2. **Get the project** and open a Command Prompt in its folder (the folder containing `api.py`).
3. **Create the virtual environment and install dependencies:**

```
python -m venv Tolis_Env
Tolis_Env\Scripts\activate.bat
python -m pip install --upgrade pip
pip install -r requirements.txt
```

The install takes a few minutes (it includes HiGHS, Pyomo, FastAPI).

Run it

```
Tolis_Env\Scripts\activate.bat
set LLM_BACKEND=ollama
uvicorn api:app --host 127.0.0.1 --port 8000
```

Then open `http://localhost:8000` in a browser. Click one of the two built-in examples and press solve.

First answer is slow. The first query loads `qwen3:14b` into memory and runs ~2–3 minutes of local inference on CPU (faster with a GPU). This is normal — it is the LLM thinking, not a hang. Subsequent queries are quicker because the model stays warm. To pre-warm before a demo, run any prompt once and wait for it to finish.

Verifying you get the same results

The numeric output is **fully deterministic and machine-independent**. The classifier and parameter extractor run at temperature 0, and the solver (HiGHS) returns the exact mathematical optimum. So the two built-in examples must produce *exactly* these answers on your Windows machine:

Built-in example	Expected result (must match exactly)
Transportation (Seattle / San Diego)	Status <code>OPTIMAL</code> , total cost \$153,675
Fixed-charge (P1/P2 → M1/M2/M3)	Status <code>OPTIMAL</code> , cost 1660 , opens routes P1 → M1, P1 → M2, P2 → M3

If the objective value and the opened routes match, the system is working correctly. The only thing that may differ between machines or runs is the *wording* of the plain-language explanation (it uses a small amount of creative temperature) — the numbers, flows, and routing decision do not change.

Quick troubleshooting

Symptom	Fix
"Ollama not running" / connection refused	Open the Ollama app, or run <code>ollama serve</code> in a separate terminal.
Empty / instant <code>{}</code> from the model	Confirm the model is <code>qwen3:14b</code> (<code>ollama list</code>). Do not substitute a different model.
<code>uvicorn</code> not found	The venv is not activated — run <code>Tolis_Env\Scripts\activate.bat</code>

first.

Port 8000 already in use

Use `--port 8765` and open that port instead.

Very slow / out of memory

Close other apps; 14B model needs ~10 GB free RAM. A CUDA GPU speeds it up dramatically.

Optimization-AI — Windows run guide. Expected values verified by direct solver run on 2026-05-16. `LLM_BACKEND=ollama` keeps everything local; no API key and no internet needed after the model is pulled.